

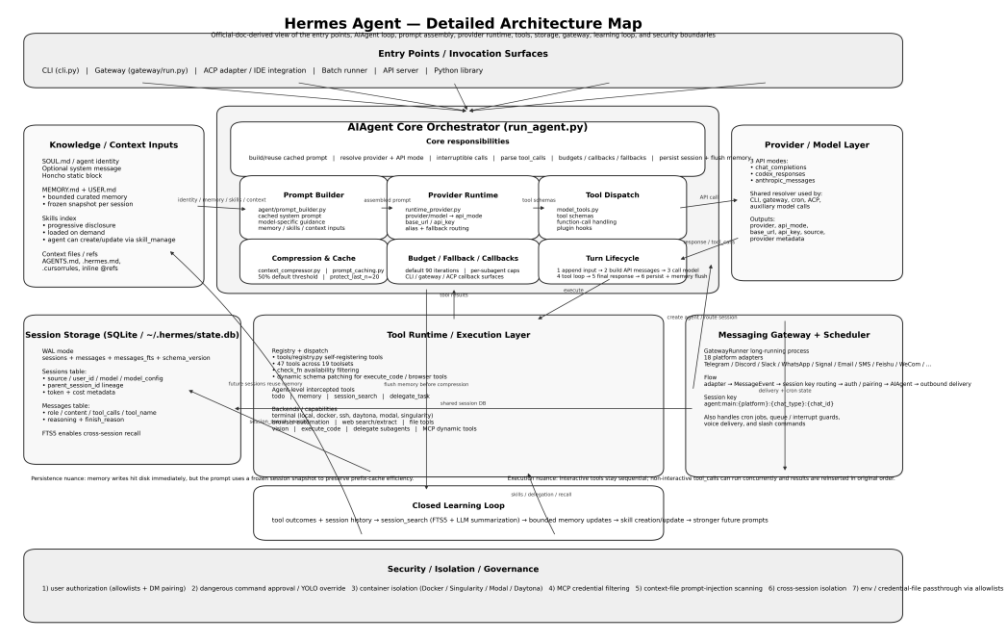
Hermes Agent

一步一步深入演讲稿

基于 Nous Research 官方网站与官方文档
从定位 → AI Agent 主循环 → Prompt/Memory/Skills →
Tool Runtime → Gateway → Security 逐层拆解

演讲主结论：Hermes 更像“带学习闭环的 Agent OS”

适合技术分享 / 方案评审 / 源码阅读导览



01 先定性：Hermes 到底是什么？

官方把它定义成“self-improving AI agent”

不是 IDE 里的 coding copilot，也不是绑死单一 API 的 chatbot wrapper；它是一个会常驻、会积累、会调度、会跨会话学习的 autonomous agent。



传统 Copilot

Hermes

Agent OS / Runtime

学习闭环 + 工具执行 + 状态持久化

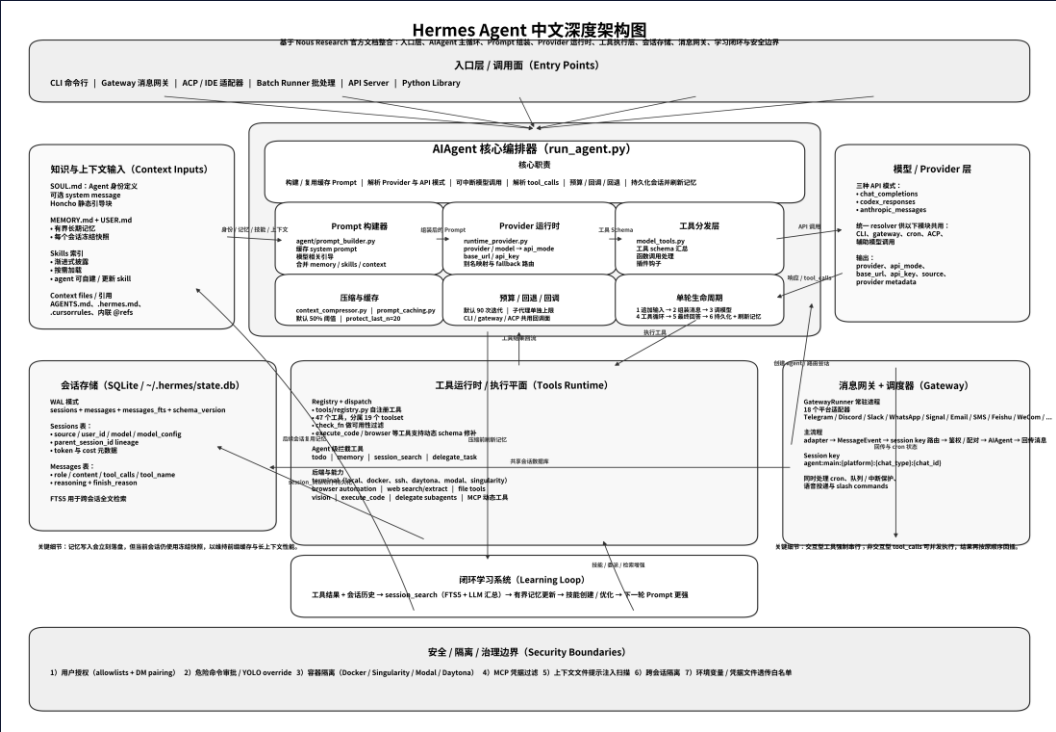
- 学习闭环：memory + skills + cross-session recall
- 运行位置：本机、Docker、SSH、Modal、Daytona、Singularity
- 入口统一：CLI / Gateway / ACP / Batch / API / Python library

02 总体架构：统一编排，不是拼装式脚手架

官方 Developer Guide 的核心信号：
所有入口最后都汇入同一个 AI Agent 编排器

- 上游：CLI、Gateway、ACP、Batch Runner、API Server、Python Library
- 中枢：run_agent.py 里的 AI Agent 统一处理 prompt、provider、tool_calls、fallback、compression
- 下游：Tool Runtime、Session Storage、Gateway、Memory/Skills/MCP 等都围绕这个主循环协同

这意味着 Hermes 的产品哲学不是“把很多 agent feature 松散拼起来”，而是围绕单一会话编排内核，构建一个可持续扩展的 agent runtime。

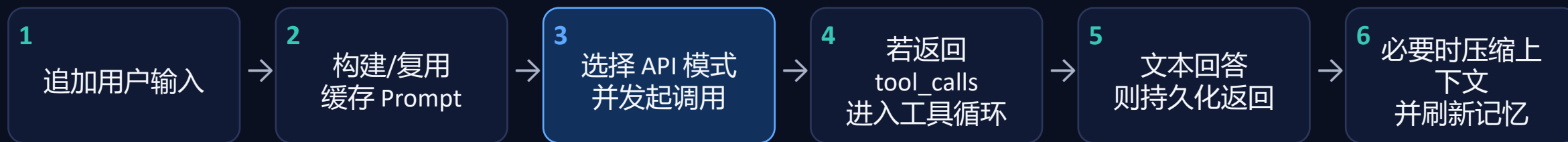


右图为我基于官网各子页面整合绘制的中文总架构图

03 大脑：AI Agent 单核 orchestrator 的工作方式

官方文档直接写明：核心编排引擎就是 `run_agent.py` 的

AI Agent。负责 prompt 装配、provider/API mode 选择、可中断模型调用、工具调度、fallback、压缩与持久化。



设计意义

- 模型接入是运行时解析问题，而不是业务层散落逻辑
- 工具执行既可串行也可并发，但结果按原顺序回插
- 当前对话、子代理、fallback provider 都由同一个编排层管控

三种 API mode

- `chat_completions`
 - `codex_responses`
 - `anthropic_messages`
- 对外 API 不同，但对内会收敛成统一的 OpenAI-style message 格式。

04 Prompt 装配：Hermes 为什么能“越用越稳”

官方最关键的设计之一：把“缓存层”和“调用时临时层”故意分开。

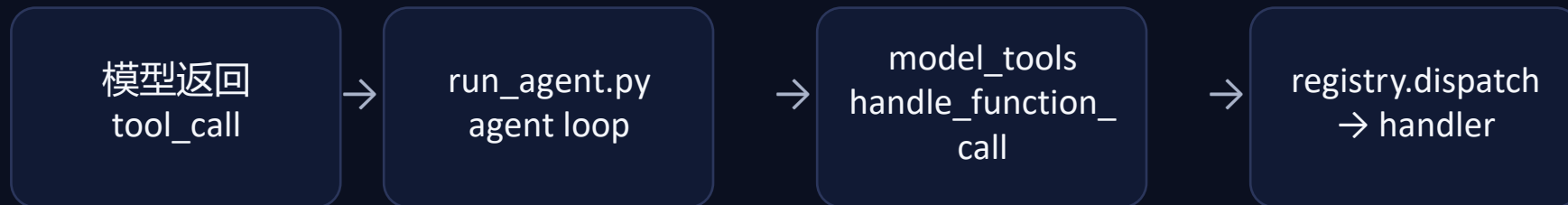
- 1 SOUL / identity
- 2 tool-aware guidance
- 3 Honcho 静态块
- 4 optional system msg
- 5 frozen MEMORY
- 6 frozen USER
- 7 skills index
- 8 context files
- 9 timestamp/session
- 10 platform hint

四个必须讲清的语义

- Memory / User Profile 是“会话开始时冻结快照”——中途写盘，不会立即污染当前稳定前缀。
- Context files 采用优先级加载：.hermes.md → AGENTS.md → CLAUDE.md → .cursorrules。
- 上下文文件在注入前会做安全扫描和截断；SOUL.md 单独走 identity 槽位。
- ephemeral_system_prompt、gateway overlay、prefill 等临时层不会持久写入 cached system prompt。

所以 Hermes 不是简单“把更多记忆塞进 prompt”。它更像一个做了缓存稳定性设计的 prompt OS。

05 执行平面：工具不是附属功能，而是操作系统接口



内建能力面

- web / browser / vision / image / TTS
- terminal / files / process / patch
- todo / clarify / execute_code / delegation
- memory / session_search / cron / delivery / MCP

4 个“被拦截”的 agent-level tools

- todo
- memory
- session_search
- delegate_task

终端后端



安全与部署策略可按 backend 切换。

06 状态脊柱：SQLite、压缩与跨会话学习闭环

state.db 是 Hermes 的状态脊柱

```
1 ~/.hermes/state.db
2 | sessions
3 | messages
4 | messages_fts
5 | schema_version
```

- WAL：支持多 reader + 单 writer
- FTS5：做跨会话全文检索
- parent_session_id：压缩后保留 lineage

Compression 触发

- Preflight：> 50% context
- Gateway auto：> 85%
- protect_last_n 默认保留 20 条

闭环学习

session_search

找回过去



memory update

沉淀事实



skills creation

沉淀流程



future prompt

强化前缀

所以“成长”并非营销词，而是由存储、检索、压缩、记忆和技能沉淀共同支撑。

07 Gateway: Hermes 为什么适合 always-on 运行

一个常驻进程，把多平台消息入口统一路由到同一套会话与编排内核。

CLI

Telegram

Discord

Slack

WhatsApp

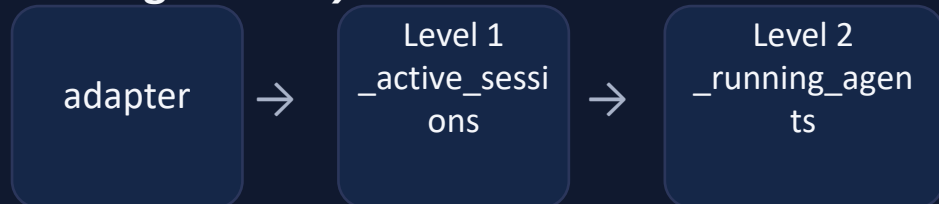
Signal

Email

SMS

Feishu

两级消息保护 (Two-Level Message Guard)



- 正在运行时，新消息先被排队并触发 interrupt event
- 特定命令（/stop /approve /deny 等）绕过普通后台任务，避免竞态
- 授权顺序：平台 allow-all → 平台 allowlist → DM pairing → 全局 allow-all → 默认拒绝

会话路由的关键： session key

```
1  
agent:main:{platform}:{chat_type}:{chat_id  
}
```

- 一个 GatewayRunner 统一管理多平台入口
- 消息、cron、语音投递、slash command 都共享这套路由骨架
- 因此 Hermes 很适合“人在 IM 上发消息，agent 在远端持续工作”的模式

08 安全模型：把边界前移，而不是只靠提示词

官方明确写的是 defense-in-depth，而且一共给了 7 层安全边界。



为什么第三层“容器隔离”最关键？

- local backend：最方便，但 agent 直接拥有当前用户文件系统权限。
- docker backend：官方强调 all capabilities dropped、no privilege escalation、PID 限制等硬化策略。
- ssh backend：推荐用于安全隔离，因为 agent 无法直接修改自己的代码。
- context files 也会被注入前扫描，防止 invisible unicode、ignore previous instructions、凭据外流等模式。

安全哲学总结：能用隔离解决的，不寄希望于提示词；需要人工确认的，进入 approval 流。

09 结论：Hermes 更像一个 Agent OS，而不是聊天壳

一句话定性

Hermes 把长期运行 agent 所需的 prompt 稳定性、工具执行、状态持久化、跨会话学习、消息网关和安全边界收束到同一 runtime。

这不是功能清单，而是一套系统设计。

最值得借鉴的 3 个模式

- 冻结快照 + 临时层分离
- 工具运行时与 agent-level state 解耦
- SQLite + FTS5 + skills 构成最小学习闭环

落地判断

- 如果你要做个人长期助手：Hermes 的设计已经很完整
- 如果你要做企业多租户大规模编排：未来可能还要把 orchestration 与 execution 拆分开来
- 但作为开源拆运行 agent substrate”，它已经非常有代表性。

Q&A